

Rapport de gestion du projet 13

Passez votre système IA du POC au MVP et réalisez votre portfolio de Data Engineer

1 Introduction

Puls-Events est une entreprise spécialisée dans la diffusion et la recommandation d'événements culturels. Sa plateforme permet aux utilisateurs de rechercher et de suivre des événements provenant notamment de sources publiques telles qu'OpenAgenda.

Dans le cadre d'un premier projet, un Proof of Concept fondé sur une architecture Retrieval-Augmented Generation a été développé afin de vérifier la faisabilité d'un assistant conversationnel capable de recommander des événements culturels à partir de questions formulées en langage naturel. Ce système s'appuie sur LangChain pour l'orchestration, sur FAISS pour la recherche vectorielle et sur les modèles Mistral pour la génération des embeddings et des réponses.

Le POC a permis de valider les principaux mécanismes du système : collecte et préparation des données OpenAgenda, découpage des contenus, vectorisation, recherche sémantique et génération de réponses contextualisées. L'évaluation fonctionnelle réalisée sur 24 questions a obtenu 22 résultats conformes, soit un taux de réussite de 91,7 %. Cependant, la solution actuelle reste une démonstration technique locale fonctionnant en ligne de commande. Elle ne dispose pas encore des fonctionnalités, des mécanismes de suivi et de l'architecture nécessaires à une utilisation à plus grande échelle.

Le Projet 13 vise donc à organiser le passage de ce POC à un Minimum Viable Product. Cette évolution doit notamment répondre à quatre besoins fonctionnels prioritaires : intégrer une mémoire conversationnelle afin d'exploiter l'historique des échanges, adapter les recommandations à la localisation de l'utilisateur, enrichir les réponses par une recherche web en temps réel et mettre en place un dispositif de monitoring permettant de suivre les performances du système ainsi que la satisfaction des utilisateurs.

La mission consiste à analyser les besoins métiers et techniques, définir le périmètre du MVP, identifier les parties prenantes, planifier les travaux, évaluer les risques, proposer une architecture cible et définir les indicateurs permettant de mesurer la réussite du projet. L'objectif n'est plus uniquement de démontrer la faisabilité technique du système RAG, mais de préparer une première version exploitable, mesurable et évolutive du produit.

2 Analyse et synthèse des besoins formulés par l'équipe

2.1 Contexte et problématique

Puls-Events exploite une plateforme web dédiée à la découverte d'événements culturels. Afin d'enrichir ce service, un premier Proof of Concept fondé sur une architecture RAG a été développé. Il collecte et prépare des données OpenAgenda, découpe les contenus en chunks, génère des embeddings avec Mistral, les indexe dans FAISS et produit des recommandations en langage naturel grâce à LangChain. Le périmètre retenu porte sur des événements récents situés à Paris et relevant principalement du spectacle vivant et de l'audiovisuel.

Le POC a démontré la faisabilité technique de la solution avec 1 336 événements nettoyés, 3 383 chunks indexés et un taux de réussite fonctionnelle de 91,7 %. Il reste toutefois limité à une exécution locale en ligne de commande, avec des traitements manuels et sans interface intégrée, gestion des sessions ni dispositif de supervision.

La problématique du projet consiste donc à organiser le passage de ce POC à un MVP exploitable. Le futur système devra conserver les acquis techniques tout en intégrant une mémoire conversationnelle, une adaptation géographique des recommandations, une recherche web en temps réel et un monitoring des performances et de la satisfaction utilisateur. L'objectif est de disposer d'une première version accessible, déployable, mesurable et suffisamment robuste pour être testée dans des conditions réelles.

2.2 Objectifs métiers

Le premier objectif métier est d'améliorer l'expérience proposée par la plateforme Puls-Events en permettant aux utilisateurs d'obtenir rapidement des recommandations culturelles pertinentes à partir de questions formulées en langage naturel. Le futur MVP devra aller au-delà de la simple recherche documentaire en tenant compte du contexte des échanges, de la localisation de l'utilisateur et, lorsque cela est nécessaire, d'informations disponibles en temps réel.

Le second objectif est de vérifier la valeur réelle du chatbot auprès des utilisateurs et des équipes produit et marketing. Le MVP doit permettre de mesurer l'intérêt du service, la pertinence des recommandations, la qualité des réponses, les temps de réponse et la satisfaction des utilisateurs. Ces indicateurs aideront Puls-Events à décider d'une éventuelle généralisation de la solution et à prioriser les évolutions futures.

Enfin, le projet doit réduire les risques liés à un déploiement à plus grande échelle en définissant une solution exploitable, évolutive et maîtrisée sur le plan technique et financier. Il s'agit de préparer une première version

suffisamment robuste pour être testée dans des conditions réelles, sans chercher à couvrir immédiatement toutes les fonctionnalités d'un produit final.

2.3 Besoins fonctionnels

Le MVP doit reprendre les fonctions principales du POC — recherche sémantique dans le catalogue et génération de recommandations en langage naturel — tout en ajoutant quatre capacités prioritaires. Le premier besoin concerne la **mémoire conversationnelle** : le chatbot doit conserver le contexte d'une session afin de comprendre les références aux échanges précédents et d'affiner progressivement ses recommandations. Le deuxième besoin porte sur la **prise en compte de la localisation** : les réponses doivent être adaptées à la position déclarée ou autorisée par l'utilisateur, ainsi qu'à un périmètre géographique configurable.

Le troisième besoin est l'intégration d'une **recherche web en temps réel**, utilisée lorsque les données de l'index sont insuffisantes ou doivent être actualisées. Cette recherche devra rester contrôlée, citer les sources consultées et éviter de mélanger des informations non vérifiées aux données du catalogue. Enfin, le MVP devra intégrer un **monitoring fonctionnel** permettant de suivre les temps de réponse, les erreurs, la pertinence des recommandations, l'utilisation des fonctionnalités et la satisfaction des utilisateurs. Ces quatre fonctions doivent permettre de proposer une expérience plus personnalisée, contextualisée et mesurable que celle du POC actuel.

2.4 Besoins non fonctionnels

Le MVP devra garantir un niveau suffisant de **performance et de disponibilité** pour offrir une expérience fluide. Les temps de réponse devront rester maîtrisés malgré l'appel à plusieurs composants — base vectorielle, modèle de langage et recherche web — et l'architecture devra pouvoir absorber une augmentation progressive du nombre d'utilisateurs sans remise en cause majeure de la solution.

La **sécurité et la confidentialité** devront être intégrées dès la conception. Les clés API et secrets devront être protégés, les échanges chiffrés et les données liées aux sessions, à l'historique conversationnel ou à la localisation limitées au strict nécessaire. Leur collecte, leur durée de conservation et leur suppression devront être définies conformément aux principes du RGPD.

Enfin, la solution devra être **maintenable, observable et reproductible**. Les composants devront être modulaires, configurables et déployables de manière automatisée. Des journaux, métriques et alertes devront permettre d'identifier les erreurs, les ralentissements et les baisses de qualité. La

documentation, les tests automatisés et le versionnement du code devront faciliter les évolutions et limiter les risques de régression.

2.5 Utilisateurs cibles et parties prenantes

Les utilisateurs cibles sont les visiteurs de la plateforme Puls-Events qui souhaitent découvrir des événements culturels à partir d'une demande formulée en langage naturel. Ils attendent des recommandations pertinentes, compréhensibles et actualisées, tenant compte de leur localisation, de leurs préférences exprimées et du contexte de la conversation. Le MVP devra rester simple d'utilisation et permettre à l'utilisateur de comprendre l'origine des informations proposées.

Les principales parties prenantes métiers sont les équipes produit et marketing, chargées d'évaluer l'intérêt du chatbot, son adoption et sa contribution à l'expérience utilisateur. **La direction ou le sponsor du projet** intervient dans la validation du périmètre, du budget et de la poursuite éventuelle du déploiement. Ces acteurs s'appuieront sur les indicateurs d'usage, de satisfaction et de qualité afin de décider des évolutions futures.

Les parties prenantes techniques regroupent les équipes Data et développement, responsables de l'architecture, du pipeline RAG, de l'intégration et des tests, ainsi que les équipes d'exploitation chargées du déploiement, du monitoring et du traitement des incidents. **Les responsables de la sécurité** et de la protection des données devront également encadrer la gestion des secrets, de l'historique conversationnel et de la localisation des utilisateurs.

2.6 Cas d'usage

Le cas d'usage principal consiste à permettre à un utilisateur de formuler une demande en langage naturel, par exemple rechercher un concert, une projection ou un événement familial, puis de recevoir une sélection d'événements pertinents issue du catalogue Puls-Events. Le chatbot doit pouvoir préciser les informations utiles, notamment le titre, le lieu, la date, une courte description et la source correspondante. Ce fonctionnement prolonge directement le système RAG validé dans le POC.

Le MVP doit également gérer des échanges successifs. Un utilisateur pourra affiner sa demande sans répéter toutes les informations déjà fournies, par exemple en ajoutant une contrainte de date, de distance ou de public. La localisation déclarée ou autorisée devra permettre de privilégier les événements proches. Lorsque les données du catalogue sont insuffisantes ou trop anciennes, le système pourra compléter la réponse par une recherche web en temps réel, en distinguant clairement les informations internes des sources externes.

Enfin, les équipes produit, marketing et techniques devront pouvoir consulter des indicateurs sur l'usage du chatbot : volume de requêtes, temps de réponse, taux d'erreur, satisfaction, questions sans résultat et recours à la recherche web. Ces informations permettront d'identifier les points de friction, d'améliorer progressivement les recommandations et d'évaluer l'intérêt du MVP avant un déploiement plus large.

2.7 Contraintes techniques et organisationnelles

Le MVP devra s'appuyer autant que possible sur les composants validés lors du POC afin de limiter les coûts, les délais et les risques de régression. L'architecture devra donc rester compatible avec Python, LangChain, Mistral et la recherche vectorielle, tout en corrigeant les dépendances locales du POC, notamment les chemins codés en dur, l'exécution manuelle des scripts et le stockage local de l'index FAISS. La recherche web en temps réel pourra faire l'objet d'une expérimentation avec smolagents, mais son intégration définitive devra être conditionnée à sa fiabilité, à sa maintenabilité et à sa compatibilité avec l'architecture cible.

La solution devra également respecter les contraintes liées à l'utilisation de services externes. Les appels aux API Mistral, OpenAgenda et aux sources web entraînent une dépendance au réseau, des limites de quotas, des coûts variables et des risques d'indisponibilité. L'architecture devra prévoir la gestion des erreurs, des délais d'attente, des reprises, de la mise en cache et des mécanismes de repli. La conservation de l'historique conversationnel et de la localisation devra par ailleurs être limitée au strict nécessaire et encadrée par des règles de sécurité, de durée de conservation et de suppression.

Sur le plan organisationnel, le projet doit rester limité au périmètre d'un MVP et respecter le budget, les délais et les compétences disponibles. Les fonctionnalités seront donc priorisées entre éléments indispensables et évolutions secondaires. La conception devra permettre aux équipes produit, marketing et techniques de suivre l'avancement, de valider progressivement les livrables et d'évaluer la valeur du chatbot avant toute industrialisation à plus grande échelle.

3 Plan de projet

3.1 Approche et hypothèses de planification

Le plan de projet présenté ci-dessous porte sur la réalisation du futur MVP de Puls-Events après validation de l'étude de design. Il correspond à une feuille de route opérationnelle permettant à une équipe projet de transformer le POC existant en une première version déployée et testable. En l'absence de date de lancement imposée, le planning est exprimé en semaines à partir d'une date de démarrage T0 correspondant à la décision de lancement du projet.

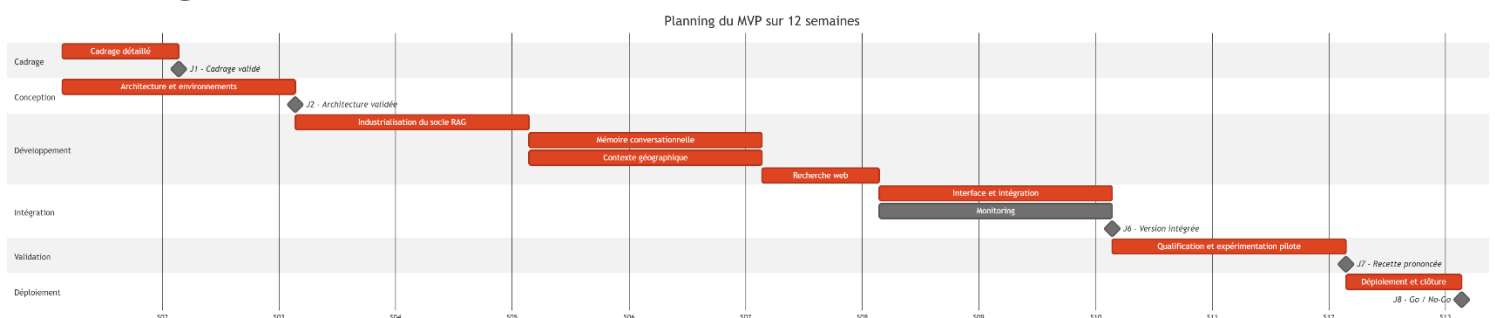
La réalisation du MVP est estimée à douze semaines. Elle repose sur une démarche itérative organisée en phases courtes, avec des validations intermédiaires par les parties prenantes. Cette approche permet de sécuriser progressivement les principaux composants : industrialisation du système RAG, déploiement cloud, mémoire conversationnelle, prise en compte de la localisation, recherche web contrôlée et monitoring. Les fonctionnalités sont intégrées et testées progressivement afin de détecter rapidement les difficultés techniques et d'éviter une validation globale trop tardive.

Le planning prévoit plusieurs points de décision permettant de vérifier que les résultats obtenus restent conformes aux besoins métiers, aux contraintes techniques et au budget. Chaque phase produit un livrable identifiable et se termine par un jalon de validation. Les éléments indispensables au fonctionnement du MVP sont traités en priorité, tandis que les fonctions secondaires pourront être reportées si elles menacent le respect des délais, des coûts ou du niveau de qualité attendu.

3.2 Jalons, échéancier et livrables

Le projet est organisé sur une durée prévisionnelle de douze semaines à compter de la décision de lancement, désignée par T0. Chaque phase se termine par un jalon de validation permettant de contrôler l'avancement, la conformité des résultats et la maîtrise des risques avant de poursuivre les travaux. Les livrables sont produits progressivement afin de disposer, à la fin du projet, d'un MVP déployé, documenté et évaluable dans des conditions proches d'un usage réel.

Diagramme de Gantt :



Description détaillée des taches :

Phase	Échéance	Travaux principaux	Jalon de validation	Livrables principaux
Lancement et cadrage détaillé	Semaine 1	Validation du périmètre, des utilisateurs cibles, des responsabilités, des critères de succès et des contraintes	J1 – Cadrage validé	Note de cadrage, périmètre du MVP, liste des parties prenantes, critères d'acceptation
Conception technique et préparation des environnements	Semaines 1 à 2	Validation de l'architecture cible, choix des services cloud, préparation des environnements, gestion des secrets et chaîne de déploiement	J2 – Architecture et socle technique validés	Schéma d'architecture, environnements de développement et de test, dépôt de code, pipeline CI/CD initial
Industrialisation du socle RAG	Semaines 3 à 4	Modularisation du POC, automatisation de la collecte et de l'indexation, exposition du moteur RAG par une API et ajout des premiers tests automatisés	J3 – Socle RAG industrialisé	Pipeline de données automatisé, index vectoriel déployable, API du chatbot, tests unitaires et d'intégration
Personnalisation des recommandations	Semaines 5 à 6	Intégration de la mémoire conversationnelle, gestion des sessions et prise en compte de la localisation de l'utilisateur	J4 – Fonctions de personnalisation validées	Module de mémoire, stockage des sessions, filtrage ou classement géographique, règles de conservation et de suppression
Recherche web en temps réel	Semaine 7	Expérimentation puis intégration contrôlée d'un mécanisme de recherche web, vérification des sources et définition des solutions de repli	J5 – Enrichissement web validé	Module de recherche web, règles de déclenchement, affichage des sources, gestion des erreurs et mécanisme de fallback
Intégration, interface et monitoring	Semaines 8 à 9	Intégration des différents composants, mise à disposition d'une interface utilisable et collecte des métriques techniques et fonctionnelles	J6 – Version intégrée disponible	Interface du chatbot, journaux applicatifs, métriques, tableau de bord de suivi et alertes principales
Qualification et expérimentation pilote	Semaines 10 à 11	Tests fonctionnels de bout en bout, tests de charge, contrôles de sécurité, évaluation de la qualité des réponses et recette avec les parties prenantes	J7 – Recette du MVP prononcée	Rapport de tests, résultats des évaluations, registre des anomalies, compte rendu de recette et liste des corrections
Déploiement et clôture	Semaine 12	Correction des anomalies bloquantes, déploiement de la version candidate, transfert des connaissances et décision de mise à disposition	J8 – Go/No-Go du MVP	MVP déployé, documentation technique et utilisateur, procédure d'exploitation, bilan du pilote et plan d'amélioration

Des démonstrations intermédiaires sont organisées à la fin des principales phases afin de recueillir les retours des équipes produit, marketing et techniques. Une anomalie bloquante concernant la sécurité, la protection des données, la stabilité du système ou la qualité minimale des recommandations peut empêcher la validation d'un jalon et entraîner une correction avant le passage à la phase suivante.

Le jalon final correspond à une décision de Go/No-Go. Le Go autorise la mise à disposition du MVP auprès d'un groupe pilote d'utilisateurs. Un Go sous réserve peut être prononcé lorsque seules des anomalies mineures subsistent et qu'elles disposent d'un plan de correction défini. Le No-Go entraîne le report du déploiement jusqu'à la résolution des problèmes considérés comme critiques.

3.3 Dépendances et chemin critique

Le respect du planning dépend de l'enchaînement cohérent des phases techniques et des validations métier. Le cadrage et l'architecture cible doivent être validés avant le démarrage des développements structurants. De même, les fonctions de personnalisation et de recherche web ne peuvent être intégrées de manière fiable qu'après la stabilisation du socle RAG, de l'API applicative et des environnements de déploiement.

Lot ou phase	Prérequis principaux	Travaux dépendants	Conséquence d'un retard
Cadrage détaillé	Validation du besoin et disponibilité des parties prenantes	Architecture, backlog et critères de recette	Décalage de l'ensemble du projet ou développement d'un périmètre inadapté
Architecture et environnements	Cadrage validé, accès au cloud et aux services externes	Industrialisation, déploiement et monitoring	Blocage des développements et des tests d'intégration
Industrialisation du socle RAG	Architecture validée, données OpenAgenda disponibles et accès à l'API Mistral	Mémoire, géolocalisation, recherche web et interface	Blocage de toutes les fonctions reposant sur la génération de recommandations
Mémoire conversationnelle	API du chatbot, stockage des sessions et règles de conservation définies	Personnalisation et tests des conversations successives	Réduction de la qualité des échanges et impossibilité de valider les cas d'usage conversationnels
Contexte géographique	Données de localisation disponibles, format géographique défini et catalogue exploitable	Classement des recommandations par proximité	Recommandations moins pertinentes et recette fonctionnelle partielle
Recherche web en temps réel	Socle RAG stable, règles de déclenchement et sources autorisées	Enrichissement des réponses et mécanisme de repli	Fonctionnalité retirée du MVP ou limitée aux données internes
Monitoring	Métriques définies et composants applicatifs instrumentés	Qualification, suivi du pilote et décision de Go/No-Go	Difficulté à mesurer la performance et à détecter les incidents
Intégration et interface	Composants prioritaires disponibles et testés séparément	Tests de bout en bout et recette	Décalage de la qualification globale
Qualification et recette	Version intégrée, jeu de tests et critères d'acceptation validés	Déploiement du MVP	Report de la mise à disposition en cas d'anomalie bloquante

Le chemin critique correspond à la succession des activités dont le retard entraînerait directement le report du jalon final :

Cadrage → architecture et environnements → industrialisation du socle RAG → fonctions prioritaires → intégration → qualification → recette → déploiement.

La stabilisation du socle RAG constitue le principal point de passage obligatoire. Tant que la collecte, l'indexation, la recherche sémantique et la génération de réponses ne sont pas automatisées et déployables, les autres fonctions ne peuvent pas être intégrées dans des conditions représentatives du futur MVP. L'intégration complète constitue le second point critique, car elle conditionne le démarrage des tests de bout en bout et de la recette.

Certaines activités peuvent néanmoins être menées en parallèle afin de limiter la durée totale. La définition des métriques, la préparation des scénarios de test, la documentation et les maquettes d'interface peuvent commencer pendant le développement du socle technique. La mémoire conversationnelle et le contexte géographique peuvent également être développés en parallèle dès que les contrats d'API et les modèles de données sont stabilisés.

Une marge de correction est intégrée aux semaines de qualification et de déploiement. Elle doit être réservée en priorité aux anomalies bloquantes relatives à la sécurité, à la perte de données, à l'indisponibilité du service ou à la qualité des recommandations. En cas de dérive importante, les fonctions classées comme secondaires dans le macro-backlog seront reportées afin de préserver le périmètre indispensable du MVP et la date du jalon final.

Les principales dépendances externes concernent enfin la disponibilité des API Mistral et OpenAgenda, l'accès aux sources web, les quotas des services cloud et la disponibilité des parties prenantes pour les validations. Des mécanismes de mise en cache, de reprise sur erreur et de repli vers les données internes devront limiter l'impact des indisponibilités techniques. Les dates de revue devront également être planifiées dès le lancement afin d'éviter qu'un retard de décision ne bloque le chemin critique.

3.4 Suivi de l'avancement et critères de validation

Le suivi du projet repose sur des points de contrôle réguliers permettant d'évaluer simultanément l'avancement, la qualité des livrables, la consommation des ressources et le niveau de risque. Cette organisation doit faciliter la détection rapide des écarts et permettre l'arbitrage entre les délais, le périmètre fonctionnel et le niveau de qualité attendu. Elle répond notamment à l'objectif de contrôler le projet en termes de délais, de coûts, de livrables et de performance.

Un point d'avancement hebdomadaire réunit le chef de projet, les référents techniques et les représentants métier concernés. Il permet d'examiner les tâches terminées, les travaux en cours, les blocages, les risques nouveaux et les décisions attendues. À la fin de chaque phase importante, une démonstration ou une revue de jalon est organisée afin de présenter les résultats aux parties prenantes et de recueillir leur validation avant la poursuite du projet.

Le pilotage s'appuie sur un tableau de bord regroupant les indicateurs suivants :

KPI	Objectif du MVP	Méthode de mesure
Temps de réponse médian	≤ 3 s	CloudWatch
Temps de réponse P95	≤ 10 s	CloudWatch
Disponibilité du service	≥ 99 %	CloudWatch
Taux d'erreurs HTTP 5xx	< 1 %	ALB / CloudWatch
Réponses contenant des sources	≥ 95 %	Évaluation RAG
Réponses pertinentes	≥ 90 %	Jeu de tests de référence
Satisfaction utilisateur	$\geq 4/5$	Feedback utilisateur
Taux de réponses sans résultat	< 5 %	Logs applicatifs
Succès du pipeline d'ingestion	≥ 99 %	EventBridge / CloudWatch
Respect du budget mensuel	± 10 % de l'estimation	AWS Cost Explorer

Chaque jalon fait l'objet d'une décision formalisée selon trois statuts : validé, validé sous réserve ou refusé. Un jalon est validé lorsque les livrables attendus sont disponibles, que les critères d'acceptation sont satisfaits et qu'aucune anomalie bloquante ne subsiste. Il peut être validé sous réserve lorsque seules des anomalies mineures restent à corriger et qu'un plan d'action est défini. Il est refusé lorsque les résultats ne permettent pas de poursuivre le projet sans risque important.

Les principaux critères de validation du MVP sont les suivants :

- le chatbot est accessible dans l'environnement cible ;
- le pipeline de collecte et d'indexation peut être exécuté automatiquement ;
- les réponses reposent prioritairement sur des sources identifiables ;
- la mémoire conversationnelle conserve les éléments utiles sans mélanger les sessions ;
- la localisation influence effectivement les recommandations proposées ;
- la recherche web est déclenchée uniquement dans les cas prévus et dispose d'un mécanisme de repli ;
- les métriques, journaux et alertes permettent de suivre le fonctionnement du système ;
- les tests fonctionnels, d'intégration, de performance et de sécurité atteignent les seuils définis ;
- aucune anomalie critique ne subsiste au moment de la recette.

En cas d'écart, une action corrective est affectée à un responsable avec une échéance et un niveau de priorité. Les écarts susceptibles d'affecter le chemin critique sont remontés immédiatement au comité de pilotage. Celui-ci peut décider de réallouer des ressources, de réduire le périmètre du MVP ou de reporter une fonctionnalité secondaire afin de préserver les exigences essentielles et la date de mise à disposition.

Le suivi se poursuit après le déploiement auprès du groupe pilote. Les retours utilisateurs, les incidents, les performances et les coûts réels sont analysés afin de mesurer la valeur apportée par le MVP et de préparer les évolutions ultérieures vers une version de production plus complète.

4 Macro backlog des fonctionnalités

4.1 Méthode de priorisation et d'estimation

Le macro backlog présente les principales fonctionnalités à développer pour transformer le POC en MVP. Il ne détaille pas encore les tâches techniques élémentaires, qui seront précisées au moment de la préparation des itérations de développement. Chaque fonctionnalité est rattachée à un domaine fonctionnel, décrite sous la forme d'un résultat attendu et évaluée selon sa priorité, sa complexité, sa charge estimée, ses principaux risques et les mesures prévues pour les limiter.

La priorisation opérationnelle du macro backlog repose sur deux catégories : « Must-Have » et « Nice-to-Have ». Les fonctionnalités « Must-Have » sont indispensables au fonctionnement et à la validation du MVP ; elles correspondent à la catégorie « Must have » de la méthode MoSCoW. Les fonctionnalités « Nice-to-Have » regroupent les éléments non indispensables au MVP, mais leur niveau de priorité peut être précisé selon MoSCoW : « Should have » pour les fonctionnalités importantes, « Could have » pour les améliorations de confort. Cette double lecture permet de conserver un backlog simple et lisible.

La complexité est évaluée selon trois niveaux — faible, moyenne ou forte — en tenant compte des dépendances techniques, des services externes, des exigences de sécurité et du niveau d'incertitude. La charge est exprimée en jours-homme et inclut la conception, le développement, les tests et la documentation de la fonctionnalité. Ces estimations sont qualitatives et devront être révisées après les expérimentations techniques, notamment pour la mémoire conversationnelle, la recherche web et le déploiement cloud.

4.2 Macro backlog des fonctionnalités

Le macro backlog des fonctionnalités est présenté dans le fichier Excel joint au rapport. Il recense les principales fonctionnalités nécessaires au passage du POC au MVP, ainsi que les évolutions envisagées pour les versions ultérieures.

Pour chaque fonctionnalité, le tableau précise son domaine, son résultat attendu, sa priorité selon la classification simplifiée « Must-Have / Nice-to-Have », sa correspondance avec la méthode MoSCoW, son niveau de complexité, sa charge estimée en jours-homme, ses principales dépendances, les risques identifiés et les mesures de mitigation associées.

Les estimations constituent une première évaluation destinée à faciliter la priorisation et la planification. Elles pourront être ajustées à mesure que les choix techniques seront confirmés et que les premières expérimentations seront réalisées.

4.3 Synthèse du périmètre MVP

Le périmètre du MVP regroupe les fonctionnalités classées « Must-Have » dans le macro backlog, soit les éléments MB-01 à MB-12. Il couvre la fiabilisation du socle RAG existant, l'accès au chatbot par une interface et une API, la mémoire conversationnelle, la prise en compte du contexte géographique, la recherche web en temps réel, l'actualisation des données événementielles, le monitoring technique et fonctionnel, la résilience, la sécurité, le déploiement cloud ainsi que l'automatisation des tests et de la livraison.

Cet ensemble représente une charge indicative de 58 jours-homme. Il constitue le périmètre minimal permettant de transformer le POC en une solution déployable, observable et testable auprès d'utilisateurs réels. Les fonctionnalités retenues doivent donc être achevées et validées avant la mise à disposition du MVP. En cas de contrainte de délai ou de ressources, les arbitrages devront porter en priorité sur le niveau de sophistication des fonctionnalités plutôt que sur leur suppression. Par exemple, la mémoire conversationnelle pourra être limitée à une session et le monitoring pourra reposer initialement sur un nombre restreint d'indicateurs.

Les fonctionnalités « Nice-to-Have », représentant 23 jours-homme supplémentaires, ne conditionnent pas la validation du MVP. Le tableau de bord métier et la personnalisation intersessions pourront être intégrés si la capacité restante le permet. Les alertes d'exploitation et l'extension géographique pourront faire l'objet d'itérations ultérieures. La création de comptes utilisateurs est explicitement reportée afin de limiter l'élargissement du périmètre, les contraintes de sécurité et les traitements supplémentaires de données personnelles.

4.4 Limites du MVP et évolutions futures

Afin de respecter les contraintes de délai, de budget et de complexité, le MVP se concentre sur les fonctionnalités apportant la plus forte valeur métier. Certaines capacités ont été volontairement écartées ou simplifiées afin de limiter les risques techniques et de garantir la livraison d'une première version exploitable :

Fonctionnalité	Décision	Justification
Personnalisation inter-sessions avancée	Reportée	Une mémoire limitée à la session est suffisante pour valider l'usage du chatbot.
Comptes utilisateurs avancés (profils, favoris)	Reportée	Non indispensable à la validation du MVP.
Déploiement multi-région	Reporté	Une seule région AWS couvre les besoins initiaux tout en réduisant les coûts.
Architecture micro services	Non retenue	Le monolithe modulaire est plus simple à développer et à maintenir pour un MVP.
Haute disponibilité complète (Multi-AZ, PRA)	Partielle	Mise en œuvre progressive selon l'adoption de la solution.
Personnalisation basée sur l'historique long terme	Reportée	Nécessite davantage de données utilisateur et soulève des contraintes RGPD supplémentaires.

Ces choix permettent de concentrer les efforts sur les fonctionnalités indispensables à la validation du produit tout en limitant les coûts et les risques du projet. Les fonctionnalités reportées constituent une feuille de route naturelle pour les futures versions, après retour d'expérience des premiers utilisateurs.

5 Architecture technique détaillée

5.1 Principes directeurs et hypothèses

L'architecture cible doit permettre de faire évoluer le système RAG existant d'un POC local vers un MVP cloud accessible à plusieurs utilisateurs. Elle conserve les principales briques fonctionnelles déjà validées — collecte des événements, nettoyage, génération d'embeddings, recherche vectorielle et génération de réponses — tout en ajoutant les capacités nécessaires à une exploitation réelle : authentification, mémoire conversationnelle, prise en compte de la localisation, recherche web en temps réel, persistance des données, supervision et déploiement automatisé.

La conception repose sur une séparation claire entre l'interface utilisateur, l'API applicative, les traitements d'ingestion, les composants du RAG et les services de stockage. Les traitements synchrones, nécessaires à la production d'une réponse, sont distingués des traitements asynchrones, comme la collecte et la réindexation des événements. Les composants applicatifs sont conteneurisés afin de garantir la reproductibilité des environnements et de faciliter leur déploiement. Les données et l'état applicatif sont conservés dans des services persistants externes aux conteneurs, afin qu'une instance puisse être remplacée ou répliquée sans perte d'information.

L'architecture privilégie des services cloud managés pour limiter la charge d'administration du MVP. Elle doit pouvoir évoluer horizontalement en augmentant le nombre d'instances applicatives lorsque le volume de requêtes augmente. Cette scalabilité reste proportionnée aux besoins du projet : l'objectif n'est pas de construire immédiatement une plateforme distribuée complexe, mais de disposer d'une architecture suffisamment modulaire pour absorber une hausse progressive de l'usage. Une approche de monolithe modulaire conteneurisé est donc retenue pour les composants métier, plutôt qu'une décomposition prématurée en nombreux micro services.

Les hypothèses structurantes suivantes sont retenues :

Hypothèse	Conséquence architecturale
Le MVP est déployé dans une seule région européenne	Réduction de la complexité et respect des contraintes de localisation des données
Le nombre d'utilisateurs reste modéré lors du lancement	Dimensionnement initial limité, avec mise à l'échelle automatique
Les composants applicatifs ne conservent pas d'état local persistant	Réplication et remplacement simplifiés des conteneurs
Les conversations doivent être mémorisées	Stockage persistant de l'historique et association à un utilisateur
La localisation peut améliorer les recommandations	Stockage d'une localisation limitée au niveau nécessaire au service
La recherche web repose sur un service externe	Isolation du fournisseur derrière une interface applicative
Les appels au modèle et aux API externes sont facturés à l'usage	Suivi des volumes, des temps de réponse et des coûts
Le catalogue d'événements doit être régulièrement actualisé	Mise en place d'un pipeline d'ingestion et d'indexation planifié
Le MVP traite principalement des données publiques et des données de compte	Mise en œuvre du chiffrement, de l'authentification et d'une gestion stricte des secrets

La sécurité, l'observabilité et la maîtrise des coûts sont intégrées dès la conception. Les communications doivent être chiffrées, les secrets centralisés dans un service dédié et les droits attribués selon le principe du moindre privilège. Les logs, métriques techniques et indicateurs de qualité du RAG doivent permettre de détecter rapidement les erreurs, les ralentissements, les réponses sans résultat et les dérives de consommation. Enfin, le recours à des standards ouverts — Docker, API HTTP, PostgreSQL et infrastructure as code — limite le couplage au fournisseur cloud et facilite une éventuelle évolution de l'architecture.

5.2 Architecture cloud scalable

L'architecture proposée doit couvrir les quatre évolutions prioritaires du MVP : mémoire conversationnelle, adaptation géographique, recherche web en temps réel et monitoring des performances. Elle doit également supprimer les principales limites du POC, notamment l'exécution locale, les traitements manuels et le stockage de l'index FAISS sur un poste unique.

L'architecture logique reste fondée sur des standards ouverts tandis que son implémentation physique repose sur des services AWS managés.

5.2.1 Vue d'ensemble

L'application est organisée en plusieurs couches afin de séparer l'accès utilisateur, les traitements applicatifs, les services externes et le stockage.

Couche	Composants proposés	Responsabilité
Accès sécurisé	Route 53, certificat TLS, AWS WAF, Application Load Balancer	Résolution DNS, chiffrement, filtrage et répartition des requêtes
Authentification	Amazon Cognito	Création des comptes, connexion et gestion des sessions
Présentation	Interface web conteneurisée	Saisie des questions et affichage des recommandations
API applicative	FastAPI sur Amazon ECS Fargate	Réception des requêtes et orchestration des traitements
Moteur RAG	LangChain, modules de mémoire, localisation et recherche web	Construction du contexte et génération des réponses
Traitements planifiés	Worker Python sur ECS Fargate, déclenché par EventBridge Scheduler	Collecte, nettoyage et réindexation des événements
Stockage objet	Amazon S3	Conservation des données brutes, nettoyées et des exports
Stockage relationnel et vectoriel	Amazon RDS PostgreSQL avec pgvector	Utilisateurs, conversations, événements, métadonnées et embeddings
Services externes	OpenAgenda, API Mistral, moteur de recherche web, service de géocodage	Sources de données et services d'intelligence artificielle
Exploitation	CloudWatch, Secrets Manager et IAM	Logs, métriques, alertes, secrets et autorisations

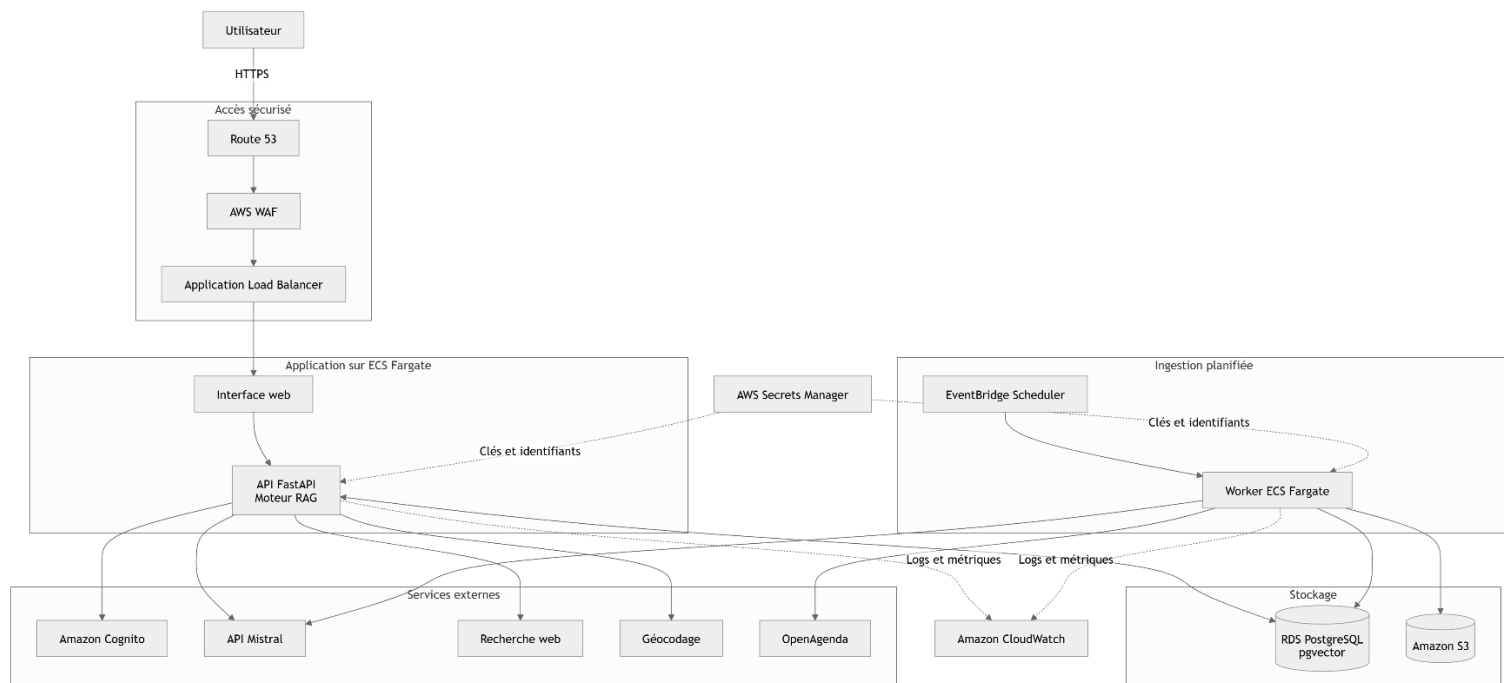
Amazon ECS est un service d'orchestration de conteneurs managé. Associé à Fargate, il permet d'exécuter les conteneurs sans administrer de serveurs ou de clusters de machines virtuelles. Le nombre de tâches applicatives peut ensuite être augmenté ou réduit automatiquement selon des métriques telles que l'utilisation du processeur, de la mémoire ou le nombre de requêtes.

L'Application Load Balancer constitue le point d'entrée de l'application et répartit les requêtes entre les différentes instances de l'API. L'architecture peut ainsi démarrer avec une seule tâche ECS, puis être étendue horizontalement sans modifier le code applicatif.

Un cluster Kubernetes (EKS) offrirait davantage de flexibilité mais augmenterait sensiblement la complexité d'administration. Pour un MVP, ECS Fargate fournit le meilleur compromis entre simplicité d'exploitation, montée en charge et maîtrise des coûts

PostgreSQL avec l'extension pgvector a été préféré à une base vectorielle spécialisée (Pinecone, Weaviate ou Milvus), car le MVP manipule un volume limité de données (3 383 embeddings dans le POC). Cette solution permet de conserver les données relationnelles, les conversations et les vecteurs dans un même moteur, réduisant ainsi la complexité d'exploitation et les coûts. Une base vectorielle dédiée pourrait être envisagée ultérieurement si le volume ou la charge devenaient significatifs.

Représentation logique simplifiée :



5.2.2 Flux conversationnel

Lorsqu'un utilisateur interroge le chatbot, la requête HTTPS est contrôlée par AWS WAF puis transmise par l'Application Load Balancer à une instance disponible de l'API FastAPI. L'identité de l'utilisateur est vérifiée par Amazon Cognito. L'API récupère ensuite dans PostgreSQL les préférences utiles, la localisation autorisée et les derniers échanges de la conversation.

Le moteur RAG transforme la question en embedding à l'aide de l'API Mistral, puis recherche dans pgvector les événements dont les représentations vectorielles sont les plus proches. Les résultats sont filtrés selon les critères métier disponibles : période, type d'événement, localisation, distance ou préférences exprimées précédemment.

Lorsque les données internes sont insuffisantes, trop anciennes ou ne permettent pas d'atteindre un seuil minimal de pertinence, le module de recherche web peut être sollicité. Les informations externes sont alors ajoutées séparément au contexte, avec leur source et leur date de consultation, afin de ne pas les confondre avec les données issues du catalogue Puls-Events.

Le contexte final est transmis au modèle de génération. La réponse produite est renvoyée à l'utilisateur, puis l'échange, les sources utilisées, le temps de traitement, le statut de la requête et les éventuels retours de satisfaction sont enregistrés. Ce fonctionnement permet à l'utilisateur d'affiner progressivement sa demande sans répéter toutes les contraintes déjà formulées.

5.2.3 Flux d'ingestion et d'indexation

L'alimentation du catalogue est indépendante du flux conversationnel. Amazon EventBridge Scheduler déclenche périodiquement une tâche ECS dédiée. Ce service permet de programmer des traitements récurrents au moyen d'expressions calendaires ou de type cron, avec des paramètres de reprise en cas d'échec.

Le pipeline applique les étapes suivantes :

1. extraction des événements depuis OpenAgenda ;
2. dépôt des données brutes horodatées dans Amazon S3 ;
3. validation, nettoyage, normalisation et déduplication ;
4. découpage des descriptions en chunks ;
5. génération des embeddings par l'API Mistral ;
6. insertion ou mise à jour des événements et vecteurs dans PostgreSQL ;
7. production de métriques et de logs d'exécution.

Chaque traitement reçoit un identifiant de lot. Les événements sont associés à un identifiant métier stable et à une version, ce qui rend les chargements idempotents : la réexécution d'un lot ne doit pas créer de doublons. La mise à jour de l'index s'effectue sans interrompre l'API conversationnelle.

Les données brutes conservées dans S3 permettent de rejouer un traitement, d'auditer une anomalie ou de reconstruire l'index sans solliciter à nouveau toutes les sources externes.

5.2.4 Stockage, sécurité et résilience

Le stockage local de l'index FAISS est remplacé par une base centralisée PostgreSQL avec l'extension pgvector. Cette solution permet aux différentes instances de l'API d'interroger le même index et de stocker conjointement les vecteurs, les événements, les utilisateurs et les conversations. Amazon RDS prend en charge PostgreSQL et l'extension pgvector, adaptée au stockage et à la recherche de vecteurs utilisés dans les architectures RAG.

La répartition proposée est la suivante :

Données	Stockage
Comptes et préférences utilisateurs	PostgreSQL
Conversations et messages	PostgreSQL
Événements et métadonnées	PostgreSQL
Chunks et embeddings	PostgreSQL avec pgvector
Données sources brutes et nettoyées	Amazon S3
Logs et métriques techniques	Amazon CloudWatch
Clés d'API et mots de passe	AWS Secrets Manager

L'Application Load Balancer est le seul composant applicatif exposé à Internet. Les conteneurs ECS et la base RDS sont placés dans des sous-réseaux privés. AWS WAF protège l'entrée HTTP contre les requêtes correspondant aux règles

de sécurité configurées et peut être directement associé à un Application Load Balancer.

Les accès entre services reposent sur des rôles IAM limités au strict nécessaire. Les secrets ne sont ni intégrés au code ni stockés dans les images Docker. Les communications utilisent TLS et les données persistantes sont chiffrées. La localisation précise n'est pas conservée lorsqu'une ville ou une zone géographique suffit à produire la recommandation.

La résilience repose enfin sur :

- les contrôles de santé de l'Application Load Balancer ;
- le redémarrage automatique des tâches ECS défaillantes ;
- la possibilité de répartir les conteneurs sur plusieurs zones de disponibilité ;
- les sauvegardes automatiques de PostgreSQL ;
- le versionnement des données sources dans S3 ;
- les délais d'attente, reprises et mécanismes de repli pour les API externes ;
- l'autoscaling de l'API lorsque la charge augmente.

Pour maîtriser le coût du MVP, le déploiement initial pourra utiliser une capacité minimale. Les options plus coûteuses, comme une base RDS Multi-AZ ou plusieurs instances applicatives permanentes, seront activées lorsque les objectifs de disponibilité ou le volume d'utilisation le justifieront.

5.3 Choix du cloud provider et veille technique

Une veille technique a été menée en juillet 2026 à partir des documentations officielles d'Amazon Web Services, Google Cloud Platform et Microsoft Azure. L'objectif n'est pas d'identifier un fournisseur universellement supérieur, mais de sélectionner celui qui répond le mieux aux besoins du MVP : exécution de conteneurs, scalabilité automatique, stockage vectoriel PostgreSQL, observabilité, sécurité, maîtrise de la complexité et continuité avec les compétences disponibles.

Comparaison des solutions

Critère	Amazon Web Services	Google Cloud Platform	Microsoft Azure
Exécution des conteneurs	ECS avec Fargate	Cloud Run	Azure Container Apps
Gestion de l'infrastructure	Exécution sans gestion des serveurs avec Fargate	Plateforme entièrement managée orientée services HTTP	Plateforme conteneurisée managée
Scalabilité	Ajustement automatique du nombre de tâches ECS selon les métriques CloudWatch	Ajustement automatique selon les requêtes, la concurrence et le CPU, avec possibilité de descendre à zéro instance	Autoscaling KEDA selon les requêtes HTTP, le CPU, la mémoire, les files de messages ou les événements
Base PostgreSQL vectorielle	Amazon RDS PostgreSQL avec pgvector	Cloud SQL PostgreSQL avec pgvector	Azure Database for PostgreSQL Flexible Server avec pgvector
Stockage objet	Amazon S3	Cloud Storage	Azure Blob Storage
Observabilité	CloudWatch et Container Insights	Cloud Monitoring et Cloud Logging	Azure Monitor et Log Analytics
Gestion des secrets	AWS Secrets Manager	Secret Manager	Azure Key Vault
Infrastructure as Code	Terraform ou AWS CloudFormation	Terraform	Terraform ou Bicep
Continuité avec les compétences du projet	Forte	Moyenne	Moyenne

AWS Fargate permet d'exécuter des conteneurs ECS sans administrer de serveurs ni de clusters de machines virtuelles. ECS Service Auto Scaling peut augmenter ou réduire automatiquement le nombre de tâches selon l'utilisation du processeur, de la mémoire ou d'autres métriques CloudWatch.

Cloud Run constitue une alternative particulièrement simple pour les applications HTTP sans état. Le service adapte automatiquement le nombre d'instances à la charge, selon l'utilisation du CPU et le nombre de requêtes concurrentes, et peut descendre à zéro instance lorsqu'aucune requête n'est reçue.

Azure Container Apps s'appuie sur KEDA pour définir des règles de mise à l'échelle basées sur les requêtes HTTP, les messages en file d'attente, le CPU, la mémoire ou différentes sources d'événements. Cette approche est particulièrement adaptée aux traitements asynchrones et événementiels.

Les trois fournisseurs répondent également au besoin de stockage vectoriel sans imposer une base spécialisée propriétaire. Amazon RDS, Google Cloud SQL et Azure Database for PostgreSQL prennent tous en charge l'extension open source pgvector, permettant de stocker et d'interroger les embeddings dans PostgreSQL.

Enfin, chacun dispose d'une solution native d'observabilité. CloudWatch Container Insights centralise les métriques et les logs des services ECS, Google Cloud Observability fournit la supervision et la journalisation de Cloud Run, tandis qu'Azure Monitor propose métriques, alertes et analyse des logs pour Azure Container Apps.

Analyse comparative

Google Cloud Platform présente l'approche la plus directe pour déployer une API conteneurisée sans état. Cloud Run réduit la quantité de configuration nécessaire et son mécanisme de mise à l'échelle jusqu'à zéro peut être intéressant pour un MVP faiblement sollicité. Cloud SQL avec pgvector couvre également le besoin de stockage relationnel et vectoriel.

Microsoft Azure offre une architecture équivalente avec Azure Container Apps et Azure Database for PostgreSQL. Son autoscaling fondé sur KEDA fournit une grande souplesse pour les traitements déclenchés par des événements. Azure serait particulièrement pertinent dans une organisation déjà intégrée à l'écosystème Microsoft.

Amazon Web Services requiert davantage de composants explicites — ECS, Fargate, Application Load Balancer, RDS, S3 et CloudWatch — mais offre une architecture très configurable et cohérente avec les besoins identifiés. Le découpage entre services ECS synchrones et tâches ponctuelles d'ingestion correspond directement à l'architecture décrite dans la section 5.2.

Décision retenue

Amazon Web Services est retenu comme fournisseur cloud pour le MVP.

Cette décision repose sur les critères suivants :

- continuité avec les compétences AWS déjà mobilisées au cours de projets précédents ;
- compatibilité directe avec les conteneurs Docker du POC ;
- intégration entre ECS Fargate, RDS PostgreSQL, S3, Secrets Manager et CloudWatch ;
- prise en charge de pgvector par Amazon RDS ;
- possibilité de mettre à l'échelle horizontalement l'API sans modifier son code ;
- séparation naturelle entre le service conversationnel permanent et les tâches d'ingestion planifiées ;
- réduction du risque d'apprentissage et du délai de mise en œuvre ;
- maintien d'une portabilité grâce à Docker, PostgreSQL, Python, FastAPI et Terraform.

Le choix d'AWS n'implique pas que GCP ou Azure soient techniquement insuffisants. Les trois plateformes peuvent satisfaire les exigences du MVP. AWS représente toutefois le meilleur compromis entre adéquation fonctionnelle, compétences disponibles, maîtrise du risque technique et capacité d'évolution.

La comparaison des coûts ne constitue pas le critère principal de cette section.

5.4 Stratégie de déploiement

La stratégie de déploiement vise à rendre les mises en production **reproductibles, automatisées et réversibles**. Elle s'appuie sur GitHub pour la gestion du code source, GitHub Actions pour l'intégration et le déploiement continu, Docker pour le packaging applicatif, Amazon ECR pour le stockage des images et Amazon ECS Fargate pour leur exécution. L'infrastructure AWS est décrite avec Terraform afin d'éviter les configurations manuelles et de conserver l'historique des modifications. Terraform permet de définir, modifier et versionner l'infrastructure au moyen de fichiers déclaratifs.

Gestion du code et des environnements

Le code applicatif, les tests, les fichiers Docker, les workflows GitHub Actions et les configurations Terraform sont regroupés dans un dépôt GitHub. Les développements sont réalisés dans des branches de courte durée, puis intégrés à la branche principale via des contrôles automatisés.

Trois environnements sont distingués :

Environnement	Usage	Mode de déploiement
Development	Développement et tests rapides en local	Docker Compose
Test	Validation technique et fonctionnelle	Déploiement automatique après intégration
Production	MVP accessible aux utilisateurs	Déploiement après validation explicite

Les environnements de test et de production utilisent des ressources, des secrets et des états Terraform séparés. Cette séparation évite qu'un test ou une modification d'infrastructure affecte directement le service en production.

Conteneurisation des composants

Deux images Docker principales sont produites :

- une image pour l'interface web ;
- une image pour le backend Python contenant l'API FastAPI et les modules métier du RAG.

L'image du backend peut également être utilisée par le worker d'ingestion en lui attribuant une commande de démarrage différente. Cette approche évite de maintenir deux images contenant les mêmes dépendances et garantit que l'API et les traitements asynchrones utilisent les mêmes composants métier.

Après leur construction, les images sont publiées dans des dépôts privés Amazon ECR. Amazon ECS peut ensuite récupérer ces images pour créer les tâches Fargate.

Chaque image est identifiée par :

- le hash du commit Git ;
- éventuellement un numéro de version pour les versions validées.

L'étiquette générique latest n'est pas utilisée en production. Une version déployée reste ainsi identifiable et peut être restaurée sans reconstruire l'application.

Pipeline d'intégration continue

À chaque pull request, GitHub Actions exécute les contrôles suivants :

- installation de l'environnement et des dépendances ;
- vérification du formatage et de la qualité du code ;
- tests unitaires ;
- tests d'intégration ;
- tests du pipeline d'ingestion et du moteur RAG ;
- analyse des dépendances et des images Docker ;
- validation de la configuration Terraform ;
- construction des images afin de vérifier leur intégrité.

Une modification ne peut être intégrée à la branche principale que si les contrôles obligatoires sont réussis.

Pipeline de déploiement continu

Après intégration dans la branche principale, le workflow construit les images définitives, les publie dans Amazon ECR puis déploie la version dans l'environnement de test. Des tests de disponibilité et des tests fonctionnels courts vérifient notamment :

- l'état de santé de l'API ;
- l'accès à PostgreSQL ;
- l'authentification ;
- l'exécution d'une requête RAG ;
- l'accessibilité des principales API externes.

Le passage en production est ensuite soumis à une validation explicite ou au marquage d'une version Git. Cette étape réduit le risque qu'une intégration technique valide déclenche automatiquement une mise en production non souhaitée.

L'authentification de GitHub Actions auprès d'AWS utilise OpenID Connect. Le workflow obtient ainsi des autorisations temporaires associées à un rôle IAM, sans stocker de clés AWS permanentes dans les secrets GitHub.

Déploiement applicatif sur ECS

Pour le MVP, la stratégie retenue est un déploiement progressif de type rolling update. Lorsqu'une nouvelle version est disponible, ECS démarre progressivement de nouvelles tâches avant de retirer les anciennes. Les contrôles de santé de l'Application Load Balancer empêchent l'envoi de trafic vers une tâche qui n'est pas encore opérationnelle.

Le mécanisme de circuit breaker d'ECS détecte les déploiements qui ne parviennent pas à atteindre un état stable. Lorsqu'un échec est détecté, le déploiement peut être interrompu et le service restauré automatiquement vers la dernière version fonctionnelle.

Un déploiement blue/green n'est pas retenu initialement, car il nécessite davantage de ressources et de configuration. Il pourra être envisagé lorsque le niveau de disponibilité attendu ou le volume d'utilisateurs justifiera le maintien simultané de deux environnements de production.

Gestion de l'infrastructure

Terraform décrit les principales ressources :

- réseau privé et groupes de sécurité ;
- Application Load Balancer ;
- services et tâches ECS Fargate ;
- dépôts Amazon ECR ;
- base Amazon RDS PostgreSQL ;
- buckets Amazon S3 ;
- rôles et politiques IAM ;
- secrets applicatifs ;
- planification EventBridge ;
- journaux, métriques et alarmes CloudWatch.

Avant toute modification, le pipeline exécute terraform plan afin de présenter les ressources qui seront créées, modifiées ou supprimées. L'application du plan de production nécessite une validation explicite. L'état Terraform est stocké à distance et séparé pour chaque environnement.

Migrations de la base de données

Les évolutions du schéma PostgreSQL sont versionnées, par exemple avec Alembic. Les migrations sont d'abord exécutées et testées en test, puis lancées en production au moyen d'une tâche ECS ponctuelle.

Les changements importants suivent autant que possible une stratégie progressive :

- ajout des nouvelles tables ou colonnes sans supprimer les anciennes ;
- déploiement du code compatible avec les deux versions ;
- migration ou recalcul des données ;
- suppression différée des structures devenues inutiles.

Cette méthode réduit le risque qu'une modification de la base rende immédiatement incompatible la version applicative précédente.

Retour arrière

La procédure de retour arrière repose sur plusieurs mécanismes :

- conservation des images Docker déjà publiées dans ECR ;
- conservation des révisions précédentes des définitions de tâches ECS ;
- rollback automatique en cas d'échec du déploiement ;
- sauvegardes automatiques de PostgreSQL ;
- versionnement ou conservation des données sources dans S3 ;
- possibilité de redéployer manuellement une version applicative antérieure.

Le retour arrière de l'application peut être automatisé. Celui de la base de données doit rester contrôlé, car la restauration d'une sauvegarde ou l'annulation d'une migration peut entraîner une perte de données créées depuis le déploiement. La priorité est donc donnée aux migrations rétrocompatibles plutôt qu'à une restauration systématique.

5.5 Stratégie de modularité

La stratégie de modularité vise à faire évoluer le MVP sans multiplier prématurément les services techniques. Le système est donc conçu comme un monolithe modulaire conteneurisé : les fonctionnalités sont regroupées dans une même application déployable, mais organisées en modules indépendants disposant de responsabilités et d'interfaces clairement définies.

Cette approche est adaptée au niveau de maturité du projet. Elle limite les coûts d'exploitation, simplifie les tests et le déploiement, tout en préparant une éventuelle séparation future de certains composants si la charge, les contraintes de disponibilité ou les rythmes d'évolution le nécessitent.

Une architecture en micro services aurait introduit des problématiques supplémentaires (communication réseau, observabilité distribuée, gestion des déploiements). Le monolithe modulaire constitue donc un compromis permettant de conserver une bonne séparation des responsabilités sans complexifier inutilement le MVP.

Découpage fonctionnel proposé

Module	Responsabilité principale
api	Exposition des endpoints FastAPI, validation des requêtes et préparation des réponses
authentication	Identification des utilisateurs et contrôle des autorisations
conversation	Gestion des sessions, messages et historiques de conversation
memory	Sélection et restitution des informations utiles issues des échanges précédents
retrieval	Recherche des événements et documents pertinents dans pgvector
generation	Construction du prompt et appel au modèle Mistral
location	Normalisation et utilisation du contexte géographique
web_search	Recherche web en temps réel et normalisation des résultats
ingestion	Collecte des événements depuis les sources externes
indexing	Nettoyage, découpage en chunks et génération des embeddings
evaluation	Tests de qualité du RAG et calcul des indicateurs de performance
observability	Journalisation, métriques, traces et suivi de la consommation
repositories	Accès à PostgreSQL, pgvector et Amazon S3
configuration	Chargement des paramètres et références vers les secrets

Le module api orchestre les traitements mais ne contient pas directement les règles métier. La recherche vectorielle, la mémoire, la géolocalisation et la génération restent isolées dans des composants spécialisés. Cette organisation évite qu'une modification d'un fournisseur externe ou d'un mode de stockage affecte l'ensemble de l'application.

Pourquoi FastAPI :

- performances ;
- documentation Swagger automatique ;
- forte adoption Python IA ;
- simplicité.

Séparation entre domaine métier et infrastructure

La logique métier doit être dissociée des implémentations techniques. Par exemple, le moteur RAG ne doit pas dépendre directement du SDK AWS, de l'API Mistral ou d'une bibliothèque particulière de recherche web. Il utilise des interfaces abstraites, dont les implémentations concrètes sont fournies au démarrage de l'application.

Cette séparation permet notamment :

- de remplacer un fournisseur de modèle sans réécrire le moteur RAG ;
- de désactiver la recherche web sans modifier les autres modules ;
- d'utiliser des implémentations simulées pendant les tests ;
- de limiter le verrouillage technologique ;
- de faire évoluer progressivement le POC vers le MVP.

Organisation du code

Une organisation possible du dépôt est la suivante :

- src/
 - api/
 - authentication/
 - conversation/
 - memory/
 - retrieval/
 - generation/
 - location/
 - web_search/
 - ingestion/
 - indexing/
 - evaluation/
 - observability/
 - repositories/
 - configuration/
- tests/
 - unit/
 - integration/
 - functional/
 - validation
- infrastructure/
 - terraform/
 - docker/
- .github/

Chaque module contient ses modèles, services, interfaces et exceptions. Les dépendances partagées sont limitées à des objets communs explicitement définis, comme les représentations d'un message, d'un événement ou d'un résultat de recherche.

Communication entre les modules

Les appels synchrones nécessaires à une réponse utilisateur restent internes à l'application. Ils ne nécessitent donc pas de communication réseau entre micro services.

Le flux conversationnel suit l'enchaînement suivant à partir de l'API :

- conversation
- memory
- location
- retrieval
- web_search éventuelle
- generation
- enregistrement de la réponse

Les traitements d'ingestion utilisent les mêmes modules métier, mais sont exécutés par un worker distinct :

- validation
- nettoyage
- indexing
- repositories

L'API et le worker peuvent utiliser la même image Docker avec des commandes de démarrage différentes. Cela mutualise le code et les dépendances tout en séparant les traitements synchrones des tâches planifiées.

Gestion de la configuration

Les paramètres variables ne sont pas intégrés dans le code. Ils sont fournis par l'environnement de déploiement :

- URLs des services externes ;
- modèles d'embedding et de génération ;
- seuils de similarité ;
- nombre maximal de résultats ;
- durée de conservation de la mémoire ;
- timeouts et nombre de tentatives ;
- activation ou non de la recherche web ;
- identifiants des ressources AWS.

Les secrets sont conservés dans AWS Secrets Manager, tandis que les paramètres non sensibles peuvent être définis dans les variables d'environnement ou dans AWS Systems Manager Parameter Store.

Testabilité des modules

Chaque module doit pouvoir être testé indépendamment. Les tests unitaires utilisent des implémentations simulées des services externes. Les tests d'intégration vérifient ensuite les échanges avec PostgreSQL, pgvector, S3 et les API partenaires.

La stratégie de tests couvre notamment :

- la sélection des événements pertinents ;
- la construction du contexte RAG ;
- la conservation et la restitution de la mémoire ;
- l'application des filtres géographiques ;
- le déclenchement conditionnel de la recherche web ;
- la gestion des indisponibilités externes ;
- l'idempotence de l'ingestion ;
- la traçabilité des sources utilisées.

Évolution vers des services indépendants

Le monolithe modulaire n'interdit pas une évolution ultérieure vers une architecture distribuée. Un module pourra être extrait lorsqu'un besoin concret le justifiera.

Situation observée	Évolution possible
Ingestion fortement consommatrice de ressources	Service ou tâche ECS autonome
Recherche web soumise à des quotas spécifiques	Service dédié avec cache et limitation de débit
Indexation volumineuse	Workers parallèles alimentés par une file de messages
Besoin de déployer l'interface indépendamment	Service frontend distinct
Forte charge sur la génération	Service de génération séparé et dimensionné indépendamment
Exigence de disponibilité élevée	Réplication et découplage des composants critiques

La séparation en microservices ne sera donc envisagée qu'en réponse à une contrainte mesurable. Cette stratégie évite d'introduire dès le MVP des problématiques supplémentaires de réseau, de sécurité interservices, de supervision distribuée et de cohérence des données.

5.6 Stratégie de monitoring et d'observabilité

La stratégie de monitoring vise à détecter rapidement les indisponibilités, les dégradations de performance et les anomalies de consommation. L'observabilité complète cette supervision en permettant d'analyser l'origine d'un incident à partir des métriques, des logs et des traces distribuées. Elle couvre l'infrastructure AWS, l'application, le pipeline d'ingestion, la qualité du système RAG et les coûts liés aux services externes.

Amazon CloudWatch constitue le point central de la supervision. CloudWatch Container Insights collecte et agrège les métriques et les journaux des clusters, services, tâches et conteneurs ECS, y compris lorsque ceux-ci sont exécutés sur AWS Fargate.

Supervision de l'infrastructure

Les métriques techniques permettent de vérifier que les ressources restent disponibles et correctement dimensionnées :

Composant	Indicateurs suivis	Objectif
Application Load Balancer	Nombre de requêtes, latence, réponses HTTP 4xx et 5xx, état des cibles	Détecter les erreurs d'accès et les tâches applicatives défaillantes
ECS Fargate	CPU, mémoire, nombre de tâches actives, redémarrages	Identifier une saturation ou une instabilité des conteneurs
RDS PostgreSQL	CPU, mémoire disponible, connexions, latence des entrées-sorties, stockage disponible	Prévenir les ralentissements et la saturation de la base
Amazon S3	Volume stocké, erreurs d'accès et nombre d'objets	Contrôler l'archivage des données d'ingestion
EventBridge et worker	Déclenchements, durée, statut et nombre d'échecs	Vérifier l'exécution régulière du pipeline
Services externes	Disponibilité, latence, quotas et taux d'erreur	Détecter une dépendance indisponible ou ralentie

Amazon RDS publie dans CloudWatch des métriques relatives à l'utilisation des ressources, à l'activité de la base et à son fonctionnement opérationnel. Ces informations permettent notamment de surveiller la charge, les connexions et l'espace de stockage.

Supervision applicative

L'API FastAPI publie des métriques personnalisées pour compléter les indicateurs fournis nativement par AWS :

- nombre de requêtes reçues ;
- taux de succès et d'erreur ;
- temps de réponse médian et percentile 95 ;
- nombre de conversations actives ;
- durée des appels au modèle Mistral ;
- durée de la recherche vectorielle ;
- durée et statut des recherches web ;
- nombre de timeouts et de nouvelles tentatives ;
- disponibilité des endpoints de contrôle de santé.

Chaque requête reçoit un identifiant de corrélation, propagé entre l'API, les modules RAG, la base de données et les appels externes. Cet identifiant permet de retrouver tous les événements associés à une même demande utilisateur.

CloudWatch accepte les métriques produites par les services AWS ainsi que des métriques personnalisées publiées par l'application, notamment à travers l'API CloudWatch ou le protocole OpenTelemetry.

Journalisation

Les applications produisent des logs structurés au format JSON afin de faciliter leur recherche et leur agrégation. Chaque entrée contient au minimum :

- la date et l'heure ;
- le niveau de gravité ;
- le composant concerné ;
- l'environnement d'exécution ;
- l'identifiant de corrélation ;
- le type d'opération ;
- la durée du traitement ;
- le statut obtenu ;
- le code d'erreur éventuel.

Les logs ne doivent contenir ni mot de passe, ni clé d'API, ni token d'authentification. Les questions et réponses des utilisateurs ne sont pas enregistrées intégralement dans les journaux techniques. Lorsque leur conservation est nécessaire pour l'évaluation du système, elles sont stockées séparément dans PostgreSQL, avec des droits d'accès et une durée de conservation adaptés.

Les niveaux de journalisation sont utilisés de manière homogène :

Niveau	Usage
DEBUG	Informations détaillées réservées au développement
INFO	Exécution normale d'une requête ou d'un traitement
WARNING	Fonctionnement dégradé, nouvelle tentative ou seuil approché
ERROR	Échec d'une opération nécessitant une analyse
CRITICAL	Indisponibilité globale ou risque d'intégrité des données

Traces distribuées

L'application est instrumentée avec OpenTelemetry afin de mesurer chaque étape importante du flux conversationnel à partir de la requête http :

- authentification
- chargement de la mémoire
- génération de l'embedding
- recherche pgvector
- recherche web éventuelle
- génération Mistral
- enregistrement de la réponse

Les traces sont envoyées à AWS X-Ray au moyen d'AWS Distro for OpenTelemetry. Cette instrumentation permet d'identifier le composant responsable d'une latence ou d'un échec, y compris lorsque la requête appelle

une base de données ou une API externe. AWS documente l'utilisation d'OpenTelemetry pour collecter les métriques et les traces des applications ECS et les transmettre à X-Ray ou CloudWatch.

Le recours à OpenTelemetry préserve également une certaine portabilité, car l'instrumentation applicative repose sur un standard ouvert plutôt que sur un format exclusivement lié à AWS.

Monitoring du pipeline d'ingestion

Chaque exécution du pipeline possède un identifiant de lot et produit un bilan comprenant :

- date et heure de début et de fin ;
- source interrogée ;
- nombre d'événements extraits ;
- nombre d'événements valides, rejetés et dédupliqués ;
- nombre de chunks créés ;
- nombre d'embeddings générés ;
- nombre de lignes insérées ou mises à jour ;
- durée de chaque étape ;
- statut final du traitement.

Un lot est considéré comme réussi uniquement si le chargement est terminé et si les contrôles de cohérence sont validés. Une absence inhabituelle de données doit être signalée, même lorsque le traitement ne produit pas d'erreur technique.

Les indicateurs suivants permettent de repérer une dérive :

- diminution anormale du volume collecté ;
- hausse du taux de rejet ;
- augmentation de la durée d'indexation ;
- différence entre le nombre de chunks et le nombre d'embeddings ;
- retard de mise à jour du catalogue ;
- échec répété d'une même source.

Suivi de la qualité du RAG

Le fonctionnement technique de l'application ne garantit pas la pertinence de ses réponses. Des indicateurs métier propres au RAG sont donc ajoutés.

Indicateur	Finalité
Nombre moyen de documents récupérés	Vérifier que le moteur trouve suffisamment de contexte
Scores de similarité	Identifier les recherches faiblement pertinentes
Taux de réponses sans résultat	Repérer un catalogue incomplet ou un seuil trop strict
Taux d'utilisation de la recherche web	Mesurer la dépendance aux données externes
Taux de réponses contenant des sources	Vérifier la traçabilité
Avis positifs et négatifs des utilisateurs	Évaluer la satisfaction réelle
Résultats du jeu de questions de référence	Détecter les régressions entre deux versions
Latence du retrieval et de la génération	Identifier les étapes limitantes
Nombre de tokens par réponse	Suivre la longueur du contexte et la consommation

Un jeu de questions de référence est exécuté avant chaque mise en production. Les résultats de la nouvelle version sont comparés à ceux de la version précédente afin d'éviter qu'une modification du prompt, de l'index ou du modèle dégrade silencieusement la qualité des réponses.

Suivi des coûts

Les services d'IA et de recherche externe étant généralement facturés à l'usage, les métriques suivantes sont enregistrées :

- nombre d'appels au modèle de génération ;
- nombre d'appels au modèle d'embedding ;
- nombre de tokens d'entrée et de sortie ;
- nombre de recherches web et de géocodages ;
- coût estimé par conversation ;
- coût moyen quotidien et mensuel ;
- volume de données stocké dans RDS, S3 et CloudWatch.

Des seuils budgétaires permettent de détecter une hausse anormale de consommation, par exemple à la suite d'une boucle applicative, d'une augmentation des requêtes automatisées ou d'un contexte RAG devenu excessivement volumineux.

Tableaux de bord et alertes

Trois tableaux de bord CloudWatch sont proposés :

1. **Disponibilité et infrastructure** : ALB, ECS, RDS et état des tâches ;
2. **Application et RAG** : latence, erreurs, retrieval, génération et recherche web ;
3. **Ingestion et coûts** : état des lots, volumes traités, tokens et dépenses estimées.

Les seuils initiaux seront ajustés après l'observation d'une période de fonctionnement normale. Les premières alertes peuvent être définies ainsi :

Alerte	Seuil initial proposé	Niveau
Taux de réponses HTTP 5xx	Supérieur à 5 % pendant 5 minutes	Critique
Latence API au percentile 95	Supérieure à 10 secondes pendant 10 minutes	Avertissement
Aucune tâche ECS saine	Pendant plus de 2 minutes	Critique
Utilisation CPU ou mémoire ECS	Supérieure à 80 % pendant 10 minutes	Avertissement
Espace disponible RDS	Inférieur à 20 %	Avertissement
Échec d'un lot d'ingestion	Un échec complet	Critique
Catalogue non actualisé	Retard supérieur à deux cycles planifiés	Avertissement
Taux d'erreur d'une API externe	Supérieur à 10 % pendant 10 minutes	Avertissement
Réponses sans résultat	Hausse significative par rapport à la référence	Avertissement
Consommation quotidienne	Dépassement du budget journalier défini	Avertissement

CloudWatch permet de créer des alarmes à partir de métriques et de déclencher une notification ou une action automatique lorsqu'un seuil est franchi. Des alarmes composites peuvent également regrouper plusieurs conditions afin de limiter les alertes non pertinentes.

Traitement des incidents

Les alertes critiques déclenchent une notification vers le canal d'exploitation retenu. La procédure de traitement comprend :

1. identification du composant concerné dans le tableau de bord ;
2. recherche des logs à partir de l'identifiant de corrélation ;
3. consultation de la trace distribuée ;
4. vérification des dépendances externes ;
5. redémarrage ou augmentation temporaire des ressources si nécessaire ;
6. rollback vers la dernière version stable lorsque l'incident est lié à un déploiement ;
7. documentation de la cause et de l'action corrective.

Après un incident significatif, un compte rendu est ajouté à la documentation du projet. Les seuils, tests ou procédures de déploiement sont ensuite adaptés afin de réduire le risque de récurrence.

6 Estimation des coûts build & OPEX

L'estimation du coût build et du coût OPEX mensuel est détaillé dans le fichier Excel joint au rapport.

Cette estimation est établie en cohérence avec l'architecture AWS que nous avons retenue (ECS Fargate, RDS PostgreSQL, S3, CloudWatch, API Web, LLM via API, etc.).

Avec cette hypothèse :

- 500/2000/5000 utilisateurs actifs ;
- 50 utilisateurs simultanés ;
- 10 000 conversations/mois.

Un troisième onglet indique des pistes d'optimisation budgétaire.

Synthèse des coûts :

- Build : **29 550 €** ;
- OPEX mensuel : **2 187 €** (pour 500 utilisateurs actifs)

7 Bilan

7.1 Rappel des étapes clés du POC au MVP

Le projet a suivi une démarche incrémentale permettant de transformer un prototype fonctionnel (Proof of Concept) en une solution de niveau MVP (Minimum Viable Product), prête à être industrialisée.

Les principales étapes ont été les suivantes :

1. Analyse du besoin métier

- identification des attentes des utilisateurs ;
- définition des fonctionnalités prioritaires ;
- cadrage des contraintes techniques et budgétaires.

2. Conception de l'architecture

- définition de l'architecture cloud cible ;
- choix des composants techniques ;
- modélisation des flux de données.

3. Organisation du projet

- élaboration du backlog ;
- planification des itérations ;
- identification et suivi des risques.

4. Conception technique

- définition des APIs et des services ;
- architecture de la mémoire conversationnelle ;
- intégration de la recherche web ;
- gestion du contexte géographique.

5. Industrialisation

- estimation des coûts Build et OPEX ;
- préparation du déploiement ;
- définition des indicateurs de suivi.

Cette démarche progressive a permis de sécuriser les choix techniques tout en conservant une vision orientée produit et valeur métier.

7.2 Justification des choix techniques et méthodologiques

Plusieurs choix structurants ont été réalisés afin de garantir la robustesse et l'évolutivité du futur système.

Sur le plan technique :

- adoption d'une architecture cloud native permettant une montée en charge horizontale ;
- séparation des responsabilités entre les services (LLM, mémoire, recherche web, géolocalisation et API) afin de faciliter la maintenance ;
- utilisation d'APIs indépendantes permettant de remplacer ou faire évoluer chaque composant sans remettre en cause l'ensemble de la plateforme ;
- prise en compte des problématiques de sécurité, de confidentialité des données et d'observabilité dès la phase de conception.

Sur le plan méthodologique :

- découpage du projet en lots fonctionnels indépendants ;
- priorisation des fonctionnalités selon leur valeur métier ;
- anticipation des risques techniques et organisationnels ;
- documentation complète des choix d'architecture afin de faciliter les futures évolutions.

Cette approche réduit les risques de développement tout en facilitant la transition vers une mise en production.

7.3 Défis rencontrés et solutions apportées

Le principal défi du projet consistait à transformer un démonstrateur technique en une architecture exploitable dans un contexte réel.

Plusieurs difficultés ont été identifiées :

Défi	Solution retenue
Persistance de la mémoire conversationnelle	Architecture dédiée avec stockage indépendant des conversations
Intégration de plusieurs services IA	Découplage via des APIs spécialisées
Coût potentiel des appels LLM	Mutualisation des traitements et optimisation des appels
Scalabilité	Infrastructure cloud basée sur des services managés
Maintenance	Architecture modulaire facilitant les évolutions futures
Pilotage du projet	Planification incrémentale avec gestion des risques et des priorités

Ces solutions permettent d'obtenir une plateforme plus robuste, plus facilement maintenable et capable d'évoluer en fonction des futurs besoins métier.

8 Conclusion

Ce projet a permis de formaliser l'ensemble des éléments nécessaires au passage d'un prototype d'intelligence artificielle vers un Minimum Viable Product (MVP) exploitable dans un environnement professionnel.

Au-delà de la définition de l'architecture technique, les travaux réalisés couvrent également la planification du projet, la priorisation des fonctionnalités, l'analyse des risques, l'estimation des coûts et la préparation du déploiement. Cette approche globale garantit une vision cohérente des aspects techniques, organisationnels et financiers.

Ce projet constitue également une synthèse des compétences développées tout au long du parcours Data Engineer OpenClassrooms, notamment en architecture de données, gestion de projet, conception de solutions cloud et industrialisation de systèmes de données et d'intelligence artificielle. Il illustre la capacité à accompagner une organisation dans la transformation d'une preuve de concept en une solution évolutive, sécurisée et prête à être déployée à plus grande échelle.

9 Annexes

9.1 Portfolio professionnel

Le portfolio professionnel réalisé dans le cadre du parcours Data Engineer OpenClassrooms est accessible publiquement à l'adresse suivante :

Portfolio GitHub Pages :

<https://ericginez.github.io/>

Le code source du portfolio est versionné dans le dépôt suivant :

Dépôt du portfolio :

<https://github.com/ericginez/ericginez.github.io>

L'ensemble des dépôts publics peut également être consulté depuis le profil GitHub :

Profil GitHub :

<https://github.com/ericginez>

Le portfolio centralise les projets techniques réalisés pendant le parcours. Chaque fiche présente, selon les éléments disponibles :

- le contexte et les objectifs du projet ;
- les besoins fonctionnels et techniques ;
- l'architecture ou le pipeline mis en œuvre ;
- les technologies utilisées ;
- les difficultés rencontrées ;
- les résultats et indicateurs obtenus ;
- les principaux livrables ;
- le lien vers le dépôt GitHub correspondant ;
- la présentation de soutenance au format PDF.

Les projets 1 à 13 sont publiés dans le portfolio professionnel.

9.2 Synthèse des projets réalisés

Projet	Sujet principal	Technologies et livrables principaux
P1	Découverte du métier de Data Engineer et cadrage du parcours	Analyse du métier, identification des compétences attendues, organisation du parcours et premiers livrables de professionnalisation
P2	Analyse comparative de systèmes éducatifs	Python, Pandas, Jupyter Notebook, analyse exploratoire, visualisations et présentation de soutenance
P3	Conception et interrogation d'une base de données immobilière	SQL, SQLite, modélisation relationnelle, base de données et requêtes métier
P4	Audit d'un environnement de données de ventes	SQL, UML, analyse des flux, diagnostic de qualité et recommandations d'évolution
P5	Migration de données médicales vers une base NoSQL	MongoDB, PyMongo, Python, Docker, migration et validation des données
P6	Prédiction de la consommation énergétique de bâtiments	Python, scikit-learn, SHAP, BentoML, Docker, modèle de régression et API de prédiction
P7	Conception et analyse d'une base de données NoSQL	MongoDB, PyMongo, Polars, Power BI, réplication et expérimentation de sharding
P8	Construction d'une infrastructure de données météorologiques	Python, Airbyte, PostgreSQL, dbt, Docker, AWS RDS, ECR et ECS
P9	Pipeline événementiel et architecture hybride	Python, Redpanda, Kafka API, Spark Structured Streaming, Docker, JSON, Parquet et architecture AWS
P10	Orchestration d'un pipeline de données commerciales	Kestra, DuckDB, Python, SQL, Docker, contrôles qualité et exports Excel/CSV
P11	Assistant conversationnel RAG pour les événements culturels	LangChain, FAISS, Mistral, OpenAgenda, embeddings, recherche sémantique et tests fonctionnels
P12	Pipeline ELT automatisé pour les avantages sportifs	Python, PostgreSQL, Kestra, pytest, Google Routes API, Slack, Power BI et architecture Bronze–Silver–Gold
P13	Passage d'un système IA du POC au MVP et réalisation du portfolio	Gestion de projet, architecture AWS, ECS Fargate, RDS PostgreSQL/pgvector, FastAPI, Terraform, CI/CD et GitHub Pages

Cette progression illustre une montée en compétence allant de l'analyse et de la modélisation des données jusqu'à la conception de pipelines automatisés, d'architectures cloud, de traitements en streaming et de systèmes d'intelligence artificielle industrialisables.

9.3 Principaux résultats et démonstrations

Les principaux résultats documentés dans le portfolio comprennent notamment :

- la conception de bases SQL et NoSQL ;
- la réalisation de pipelines batch, incrémentaux et événementiels ;
- l'orchestration de traitements avec Kestra ;
- la mise en œuvre d'architectures Bronze–Silver–Gold ;
- le déploiement de composants de données dans AWS ;
- la création de rapports Power BI ;
- la construction d'un système RAG fondé sur LangChain, FAISS et Mistral ;
- la mise en place de tests automatisés et de contrôles de qualité ;
- la documentation d'architectures locales, cloud et hybrides.

Pour le système RAG servant de point de départ au Projet 13, les principaux résultats du POC sont :

Indicateur	Résultat
Événements culturels retenus	1 336
Chunks documentaires générés	3 383
Vecteurs indexés dans FAISS	3 383
Questions du jeu d'évaluation	24
Réponses conformes	22
Taux de réussite	91,70%

Ces résultats démontrent la faisabilité technique du système initial et justifient la préparation de son passage vers un MVP déployable, observable et utilisable par plusieurs utilisateurs.

9.4 Livrables et ressources complémentaires du Projet 13

Les documents suivants complètent le présent rapport de gestion :

Livrable	Contenu
Macro backlog des fonctionnalités	Fonctionnalités du MVP, priorités Must-Have/Nice-to-Have, correspondance MoSCoW, complexité, charge, dépendances, risques et mesures de mitigation
Estimation des coûts build et OPEX	Coût du développement initial, estimation mensuelle des charges AWS et des services externes, scénarios de fréquentation et pistes d'optimisation
Présentation de soutenance du Projet 13	Synthèse du contexte, du passage du POC au MVP, du planning, de l'architecture cible, des coûts, des risques et du bilan
Portfolio professionnel	Présentation publique des projets, technologies, compétences, résultats et liens vers les dépôts
Dépôt du POC RAG	Scripts d'extraction, chunking, vectorisation, index FAISS, chatbot, tests et documentation

9.5 Références principales

Portfolio et profil :

- Portfolio professionnel : <https://ericginez.github.io/>
- Profil GitHub : <https://github.com/ericginez>

Dépôts directement liés au Projet 13 :

- Projet 13 - Passage du POC au MVP : <https://github.com/ericginez/rag-poc-to-mvp-project>
- POC RAG du Projet 11 : <https://github.com/ericginez/RAG-chatbot>
- Pipeline ELT du Projet 12 : <https://github.com/ericginez/sports-benefits-data-pipeline>
- Dépôt source du portfolio : <https://github.com/ericginez/ericginez.github.io>

Les liens vers les autres dépôts sont centralisés dans les fiches correspondantes du portfolio.
